

Below is a complete list of ReactJS coding questions commonly asked in interviews — from basic → intermediate → advanced, along with answers/code snippets. This is perfect for fresher / junior React developer interviews.

✅ ReactJS Coding Interview Questions with Answers

◆ 1. Create a Counter Component (useState)

Question: Build a counter with + and – buttons.

✓ Answer:

```
jsx Copy code

import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>{count}</h2>
      <button onClick={() => setCount(count + 1)}>+</button>
      <button onClick={() => setCount(count - 1)}>-</button>
    </div>
  );
}

export default Counter;
```

◆ 2. Controlled Input Component

Question: Create an input whose value updates live.

✓ Answer:

```
jsx Copy code

function InputDemo() {
  const [text, setText] = useState("");

  return (
    <>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <p>You typed: {text}</p>
    </>
  );
}
```

◆ 3. Render a List using .map()

Question: Display ["Apple", "Banana", "Mango"] on screen.

✓ Answer:

```
jsx Copy code

function Fruits() {
  const fruits = ["Apple", "Banana", "Mango"];
```

```
);  
}
```

◆ 3. Render a List using `.map()`

Question: Display `["Apple", "Banana", "Mango"]` on screen.

✓ Answer:

jsx

Copy code

```
function Fruits() {  
  const fruits = ["Apple", "Banana", "Mango"];  
  
  return (  
    <ul>  
      {fruits.map((fruit, index) => (  
        <li key={index}>{fruit}</li>  
      ))}  
    </ul>  
  );  
}
```

◆ 4. Show/Hide Toggle Component

Question: Toggle a paragraph when button is clicked.

✓ Answer:

jsx

Copy code

```
function ToggleText() {  
  const [show, setShow] = useState(false);  
  
  return (  
    <>  
      <button onClick={() => setShow(!show)}>  
        {show ? "Hide" : "Show"}  
      </button>  
  
      {show && <p>Hello, this is visible now!</p>}  
    </>  
  );  
}
```

◆ 5. Form Submission in React

Question: Create a form and show entered data in alert.

✓ Answer:

jsx

Copy code

```
function MyForm() {  
  const [name, setName] = useState("");  
  const [email, setEmail] = useState("");  
  
  const handleSubmit = (e) => {  
    e.preventDefault();  
    alert(`Name: ${name}, Email: ${email}`);  
  };  
}
```

```

    </>
  );
}

```

◆ 5. Form Submission in React

Question: Create a form and show entered data in alert.

✓ Answer:

jsx

[Copy code](#)

```

function MyForm() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    alert(`Name: ${name}, Email: ${email}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input placeholder="Name" onChange={(e) => setName(e.target.value)} />
      <input placeholder="Email" onChange={(e) => setEmail(e.target.value)} />
      <button type="submit">Submit</button>
    </form>
  );
}

```

◆ 6. Pass Data using Props

Question: Parent to child data passing.

✓ Answer:

jsx

[Copy code](#)

```

function Child({ msg }) {
  return <h3>{msg}</h3>;
}

function Parent() {
  return <Child msg="Hello from Parent!" />;
}

```

◆ 7. Conditional Rendering

Question: Show "Welcome User" if logged in, else "Please Login".

✓ Answer:

jsx

[Copy code](#)

```

function UserStatus({ isLoggedIn }) {
  return <h2>{isLoggedIn ? "Welcome User" : "Please Login"}</h2>;
}

```

◆ 8. Fetch API Data (useEffect)

```
return <Child msg="Hello from Parent!" />;
}
```

◆ 7. Conditional Rendering

Question: Show "Welcome User" if logged in, else "Please Login".

✓ Answer:

jsx

[Copy code](#)

```
function UserStatus({ isLoggedIn }) {
  return <h2>{isLoggedIn ? "Welcome User" : "Please Login"}</h2>;
}
```

◆ 8. Fetch API Data (useEffect)

Question: Fetch posts from API and display.

✓ Answer:

jsx

[Copy code](#)

```
import { useEffect, useState } from "react";

function FetchPosts() {
  const [posts, setPosts] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/posts")
      .then(res => res.json())
      .then(data => setPosts(data));
  }, []);

  return (
    <>
      {posts.slice(0, 5).map(post => (
        <p key={post.id}>{post.title}</p>
      ))}
    </>
  );
}
```

◆ 9. Reusable Button Component

Question: Create a button component that accepts text + click handler.

✓ Answer:

jsx

[Copy code](#)

```
function MyButton({ text, onClick }) {
  return <button onClick={onClick}>{text}</button>;
}
```

◆ 10. useEffect Component Mount

Question: Log message when component loads.

}

◆ 9. Reusable Button Component

Question: Create a button component that accepts text + click handler.

✓ Answer:

jsx

Copy code

```
function MyButton({ text, onClick }) {  
  return <button onClick={onClick}>{text}</button>;  
}
```

◆ 10. useEffect Component Mount

Question: Log message when component loads.

✓ Answer:

jsx

Copy code

```
useEffect(() => {  
  console.log("Component Mounted");  
}, []);
```

◆ 11. Simple Routing (React Router)

✓ Answer:

jsx

Copy code

```
import { BrowserRouter, Routes, Route } from "react-router-dom";  
  
function App() {  
  return (  
    <BrowserRouter>  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/about" element={<About />} />  
      </Routes>  
    </BrowserRouter>  
  );  
}
```

◆ 12. Highlight Selected List Item

✓ Answer:

jsx

Copy code

```
function HighlightList() {  
  const [selected, setSelected] = useState(null);  
  const items = ["One", "Two", "Three"];  
  
  return (  
    <>  
      {items.map((item, index) => (  
        <p>
```

```

    </Routes>
  </BrowserRouter>
);
}

```

◆ 12. Highlight Selected List Item

✓ Answer:

jsx

[Copy code](#)

```

function HighlightList() {
  const [selected, setSelected] = useState(null);
  const items = ["One", "Two", "Three"];

  return (
    <>
      {items.map((item, index) => (
        <p
          key={index}
          onClick={() => setSelected(index)}
          style={{
            background: selected === index ? "lightblue" : "white"
          }}
        >
          {item}
        </p>
      ))}
    </>
  );
}

```

◆ 13. Todo List

✓ Answer:

jsx

[Copy code](#)

```

function Todo() {
  const [task, setTask] = useState("");
  const [list, setList] = useState([]);

  const addTask = () => {
    setList([...list, task]);
    setTask("");
  };

  return (
    <>
      <input value={task} onChange={(e) => setTask(e.target.value)} />
      <button onClick={addTask}>Add</button>

      <ul>
        {list.map((t, i) => (
          <li key={i}>{t}</li>
        ))}
      </ul>
    </>
  );
}

```

}

◆ 13. Todo List

✓ Answer:

jsx

Copy code

```
function Todo() {
  const [task, setTask] = useState("");
  const [list, setList] = useState([]);

  const addTask = () => {
    setList([...list, task]);
    setTask("");
  };

  return (
    <>
      <input value={task} onChange={(e) => setTask(e.target.value)} />
      <button onClick={addTask}>Add</button>

      <ul>
        {list.map((t, i) => (
          <li key={i}>{t}</li>
        ))}
      </ul>
    </>
  );
}
```

◆ 14. Timer with setInterval

✓ Answer:

jsx

Copy code

```
function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds(prev => prev + 1);
    }, 1000);

    return () => clearInterval(interval);
  }, []);

  return <h2>{seconds} seconds</h2>;
}
```

◆ 15. Change Background Color

✓ Answer:

jsx

Copy code

```
function BgChanger() {
  const [color, setColor] = useState("white");
```



```

    <li key={i}>{t}</li>
  )}
</ul>
</>
);
}

```

◆ 14. Timer with setInterval

✓ Answer:

```

jsx
Copy code

function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds(prev => prev + 1);
    }, 1000);

    return () => clearInterval(interval);
  }, []);

  return <h2>{seconds} seconds</h2>;
}

```

◆ 15. Change Background Color

✓ Answer:

```

jsx
Copy code

function BgChanger() {
  const [color, setColor] = useState("white");

  return (
    <div style={{ backgroundColor: color, padding: "20px" }}>
      <button onClick={() => setColor("yellow")}>Yellow</button>
      <button onClick={() => setColor("pink")}>Pink</button>
    </div>
  );
}

```

🔥 Do you also want?

- ✓ Medium + Advanced React coding questions (Redux, Context API, performance)
- ✓ React interview MCQs
- ✓ React system design questions (e.g., how to design Instagram feed in React)

Just tell me — I can generate the full interview pack! ↓






1. Lift State Up (Child → Parent data pass)

Question:

Create two child inputs. When both change, update the parent and display the combined value.

✓ Answer:

jsx

Copy code

```
function Child({ setValue }) {
  return <input onChange={(e) => setValue(e.target.value)} />;
}

function Parent() {
  const [first, setFirst] = useState("");
  const [second, setSecond] = useState("");

  return (
    <>
      <Child setValue={setFirst} />
      <Child setValue={setSecond} />
      <h3>{first + " " + second}</h3>
    </>
  );
}
```

2. Search Filter in List

Question: Filter names as you type.

✓ Answer:

jsx

Copy code

```
function SearchList() {
  const items = ["Ankit", "Rohan", "Sita", "Priya"];
  const [query, setQuery] = useState("");

  const filtered = items.filter((item) =>
    item.toLowerCase().includes(query.toLowerCase())
  );

  return (
    <>
      <input onChange={(e) => setQuery(e.target.value)} />
      {filtered.map((i) => (
        <p key={i}>{i}</p>
      ))}
    </>
  );
}
```



+ Ask anything



}

5. Pagination Component

✓ Answer:

jsx

[Copy code](#)

```
function Pagination({ data, itemsPerPage }) {
  const [page, setPage] = useState(1);

  const start = (page - 1) * itemsPerPage;
  const end = start + itemsPerPage;

  return (
    <>
      {data.slice(start, end).map((item) => (
        <p>{item}</p>
      ))}

      <button onClick={() => setPage(page - 1)}>Prev</button>
      <button onClick={() => setPage(page + 1)}>Next</button>
    </>
  );
}
```

Advanced React Coding Interview Questions (with Answers)

1. Build a Custom Hook (useFetch)

✓ Answer:

jsx

[Copy code](#)

```
function useFetch(url) {
  const [data, setData] = useState(null);

  useEffect(() => {
    fetch(url)
      .then((res) => res.json())
      .then((d) => setData(d));
  }, [url]);

  return data;
}
```



2. Context API (Global State)

+ Ask anything



2. Context API (Global State)

✓ Answer:

```
jsx Copy code

const UserContext = createContext();

function App() {
  return (
    <UserContext.Provider value={"Ankit"}>
      <Profile />
    </UserContext.Provider>
  );
}

function Profile() {
  const user = useContext(UserContext);
  return <h2>Hello {user}</h2>;
}
```

3. Redux Basic Example

✓ Answer:

Reducer

```
jsx Copy code

const counterReducer = (state = 0, action) => {
  switch (action.type) {
    case "INC": return state + 1;
    case "DEC": return state - 1;
    default: return state;
  }
};
```

Dispatch

```
jsx Copy code

const dispatch = useDispatch();

<button onClick={() => dispatch({ type: "INC" })}>+</button>
```

4. Memoization (React.memo + useMemo + useCallback)

✓ React.memo:



```
jsx Copy code

+ Ask anything
```

4. Memoization (React.memo + useMemo + useCallback)

✓ React.memo:

jsx

Copy code

```
const Child = React.memo(({ value }) => {
  console.log("Rendered");
  return <h3>{value}</h3>;
});
```

✓ useMemo (expensive calculation):

jsx

Copy code

```
const calc = useMemo(() => heavyCalculation(num), [num]);
```

✓ useCallback:

jsx

Copy code

```
const handleClick = useCallback(() => {
  console.log("Clicked");
}, []);
```

5. Error Boundary

✓ Answer:

jsx

Copy code

```
class ErrorBoundary extends React.Component {
  state = { hasError: false };

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  render() {
    return this.state.hasError ? <h2>Something went wrong</h2> : this.props.children;
  }
}
```

6. Higher-Order Component (HOC)

✓ Answer:

jsx

Copy code

```
function withLogger(Component) {
```

+ Ask anything



6. Higher-Order Component (HOC)

✓ Answer:

jsx

Copy code

```
function withLogger(Component) {
  return function Wrapper(props) {
    console.log("Rendering component with props", props);
    return <Component {...props} />;
  };
}
```

7. Infinite Scroll Component

✓ Answer:

jsx

Copy code

```
function InfiniteScroll() {
  const [items, setItems] = useState([...Array(20).keys()]);

  useEffect(() => {
    const handleScroll = () => {
      if (window.innerHeight + window.scrollY >= document.body.offsetHeight) {
        setItems((prev) => [...prev, ...Array(20).keys()]);
      }
    };

    window.addEventListener("scroll", handleScroll);
    return () => window.removeEventListener("scroll", handleScroll);
  }, []);

  return items.map((i) => <p key={i}>Item {i}</p>);
}
```

8. Lazy Loading & Suspense

✓ Answer:

jsx

Copy code

```
const LazyComponent = React.lazy(() => import("./Demo"));

function App() {
  return (
    <Suspense fallback=<h2>Loading...</h2>>
      <LazyComponent />
    </Suspense>
  );
}
```

+ Ask anything



8. Lazy Loading & Suspense

✓ Answer:

```
jsx Copy code

const LazyComponent = React.lazy(() => import("./Demo"));

function App() {
  return (
    <Suspense fallback=<h2>Loading...</h2>>
      <LazyComponent />
    </Suspense>
  );
}
```

9. Optimize React App Performance

Using:

- React.memo
- useCallback
- useMemo
- Virtualization (react-window)
- Lazy loading
- Code splitting
- Avoid unnecessary re-renders
- Use keys properly

10. Build a Custom Form Hook

✓ Answer:

```
jsx Copy code

function useForm(initial) {
  const [form, setForm] = useState(initial);

  const handleChange = (e) =>
    setForm({ ...form, [e.target.name]: e.target.value });

  return { form, handleChange };
}
```



Want more?

I can give you:



+ Ask anything





```
import styled from 'styled-components'

const Button = styled.div`
  width : 100px ;
  cursor : pointer ;
  text-decoration : none;
`

export default Button;
```

43. What is prop drilling and its disadvantages?

Prop drilling is basically a situation when the same data is being sent at almost every level due to requirements in the final level. The problem with Prop Drilling is that whenever data from the Parent component will be needed, it would have to come from each level, Regardless of the fact that it is not needed there and simply needed in last.

44. What is conditional rendering in React?

Conditional rendering in React is used when you want to render different UI elements based on certain conditions. For example, rendering a login button if a user is not logged in or rendering a

types in the form fields.

- **Submit the Form:** Handle the form submission with an onSubmit event, and prevent the default

Open In App

8:22

11 KB/s 5G 61%

React Course React Tutorial React Exercise React Basic Concepts React Components React Props React Hook Sign In

buttons. React provides controlled and uncontrolled components for handling form elements, allowing developers to manage user input and form state in an efficient and structured way.

66. How to create forms in React?

Creating forms in React involves handling user input and managing the form's state. In React, form elements like `<input>`, `<textarea>`, and `<select>` are controlled components, meaning their values are controlled by the React state.

- **Create the State for the Form:** Use the `useState` hook to manage form input values.
- **Set the Value of Form Elements:** Bind the form elements' value attribute to the corresponding state variable to make the inputs controlled.
- **Handle User Input:** Use an `onChange` event handler to update the state whenever the user types in the form fields.
- **Submit the Form:** Handle the form submission with an `onSubmit` event, and prevent the default behavior to stop the page from refreshing.

```
import React, { useState } from 'react';

function MyForm() {
  // State to store input values
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [message, setMessage] = useState('');

  // Handle input change for each field
  const handleNameChange = (e) => setName(e.target.value);
  const handleEmailChange = (e) => setEmail(e.target.value);
  const handleMessageChange = (e) => setMessage(e.target.value);

  // Handle form submission
  const handleSubmit = (e) => {
    e.preventDefault(); // Prevent default form submission behavior
    console.log('Form submitted:', { name, email, message });
  };

  return (
    <form onSubmit={handleSubmit}>
      <div>
        <label>Name:</label>
        <input type="text" value={name} onChange={handleNameChange} />
      </div>
      <div>
        <label>Email:</label>
        <input type="email" value={email} onChange={handleEmailChange} />
      </div>
      <div>
        <label>Message:</label>
        <textarea value={message} onChange={handleMessageChange} />
      </div>
      <button type="submit">Submit</button>
    </form>
  );
}

export default MyForm;
```

Output:

Name:

Email:

Message:

Submit

React Form

67. What is Lazy Loading in React?

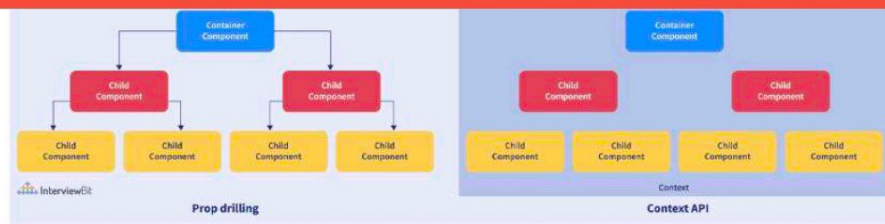
[Lazy Loading](#) in React is a technique used to load components only when they are needed, instead of loading everything at once when the app starts. This helps improve the performance of the app by reducing the initial loading time.

In React, `React.lazy()` is used to implement lazy loading for components, which allows you to split your code into smaller bundles and load them only when required.

68. How is React different from React Native?

React	React Native

< ReactJS Tutorial



Sometimes while developing React applications, there is a need to pass data from a component that is higher in the hierarchy to a component that is deeply nested. To pass data between such components, we pass props from a source component and pass the prop to the next component in the hierarchy till we reach the deeply nested component.

The **disadvantage** of using prop drilling is that the components that should otherwise be not aware of the data have access to it.

14. What are error boundaries?

Introduced in version 16 of React, Error boundaries provide a way for us to catch errors that occur in the render phase.

• What is an error boundary?

Any component which uses one of the following lifecycle methods is considered an error boundary.

In what places can an error boundary detect an error?

1. Render phase
2. Inside a lifecycle method
3. Inside the constructor

Without using error boundaries:

```
class CounterComponent extends React.Component{
  constructor(props){
    super(props);
    this.state = {
      counterValue: 0
    }
    this.incrementCounter = this.incrementCounter.bind(this);
  }
  incrementCounter(){
    this.setState(prevState => {counterValue: prevState.counterValue + 1});
  }
  render(){
    if(this.state.counterValue === 2){
      throw new Error('Crashed');
    }
    return(
      <div>
        <button onClick={this.incrementCounter}>Increment Value</button>
        <p>Value of counter: {this.state.counterValue}</p>
      </div>
    )
  }
}
```

In the code above, when the counterValue equals 2, we throw an error inside the render method.

When we are not using the error boundary, instead of seeing an error, we see a blank page. Since any error inside the render method leads to unmounting of the component. To display an error that occurs inside the render method, we use error boundaries.

With error boundaries: As mentioned above, error boundary is a component using one or both of the following methods: `getDerivedStateFromError` and `componentDidCatch`.

Let's create an error boundary to handle errors in the render phase:

```
class ErrorBoundary extends React.Component {
  constructor(props) {
```



Rotate screen



Play



Share



Search

<button onClick={() => handleClick('DemoButton')}>To Abc

[React Router Interview Questions](#)[Angular Interview Questions and Answers](#)

22. How to pass data between sibling components using React router?

Passing data between sibling components of React is possible using React Router with the help of `history.push` and `match.params`.

In the code given below, we have a Parent component `AppDemo.js` and have two Child Components `HomePage` and `AboutPage`. Everything is kept inside a Router by using React-router Route. It is also having a route for `/about/{params}` where we will pass the data.

```
import React, { Component } from 'react';
class AppDemo extends Component {
  render() {
    return (
      <Router>
        <div className="AppDemo">
          <ul>
            <li>
              <NavLink to="/" activeStyle={{ color: 'blue' }}>Home</NavLink>
            </li>
            <li>
              <NavLink to="/about" activeStyle={{ color: 'blue' }}>About
            </li>
          </ul>
          <Route path="/about/:aboutId" component={AboutPage} />
          <Route path="/about" component={AboutPage} />
          <Route path="/" component={HomePage} />
        </div>
      </Router>
    );
  }
}
export default AppDemo;
```

The `HomePage` is a functional component with a button. On button click, we are using `props.history.push('/about/' + data)` to programmatically navigate into `/about/data`.

```
export default function HomePage(props) {
  const handleClick = (data) => {
    props.history.push('/about/' + data);
  }
  return (
    <div>
      <button onClick={() => handleClick('DemoButton')}>To About</button>
    </div>
  )
}
```

Also, the functional component `AboutPage` will obtain the data passed by `props.match.params.aboutId`.

```
export default function AboutPage(props) {
  if(!props.match.params.aboutId) {
    return <div>No Data Yet</div>
  }
  return (
    <div>
      {`Data obtained from HomePage is ${props.match.params.aboutId}`}
    </div>
  )
}
```

After button click in the `HomePage` the page will look like below:



Share



Edit

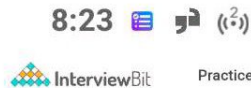


Delete



More

23. How to re-render the view when the browser is resized?



23. How to re-render the view when the browser is resized?

It is possible to listen to the resize event in `componentDidMount()` and then update the width and height dimensions. It requires the removal of the event listener in the `componentWillUnmount()` method.

Using the below-given code, we can render the view when the browser is resized.

```
class WindowSizeDimensions extends React.Component {
  constructor(props){
    super(props);
    this.updateDimension = this.updateDimension.bind(this);
  }

  componentWillMount() {
    this.updateDimension()
  }
  componentDidMount() {
    window.addEventListener('resize', this.updateDimension)
  }
  componentWillUnmount() {
    window.removeEventListener('resize', this.updateDimension)
  }
  updateDimension() {
    this.setState({width: window.innerWidth, height: window.innerHeight})
  }
  render() {
    return <span>{this.state.width} x {this.state.height}</span>
  }
}
```

24. How to create a switching component for displaying different pages?

A switching component refers to a component that will render one of the multiple components. We should use an object for mapping prop values to components.

A below-given example will show you how to display different pages based on page prop using switching component:

```
import HomePage from './HomePage'
import AboutPage from './AboutPage'
import FacilitiesPage from './FacilitiesPage'
import ContactPage from './ContactPage'
import HelpPage from './HelpPage'
const PAGES = {
  home: HomePage,
  about: AboutPage,
  facilities: FacilitiesPage,
  contact: ContactPage,
  help: HelpPage
}
const Page = (props) => {
  const Handler = PAGES[props.page] || HelpPage
  return <Handler {...props} />
}
// The PAGES object keys can be used in the prop types for catching errors during dev-time.
Page.propTypes = {
  page: PropTypes.oneOf(Object.keys(PAGES)).isRequired
}
```

25. Explain how to create a simple React Hooks example program.

I will assume that you are having some coding knowledge about JavaScript and have installed Node on your system for creating a below given React Hook program. An installation of Node comes along with the command-line tools: npm and npx, where npm is useful to install the packages into a project and npx is useful in running commands of Node from the command line. The npx looks in the current project folder for checking whether a command has been installed there. When the command is not available on your computer, the npx will look in the npmjs.com repository, then the latest version of the command script will be loaded and will run without locally installing it. This feature is useful in creating a skeleton React application within a few key presses.

Open the Terminal inside the folder of your choice, and run the following command:

```
npx create-react-app react-items-with-hooks
```

Here, the `create-react-app` is an app initializer created by Facebook, to help with the easy and quick creation of React application, providing options to customize it while creating the application? The above command will create a new folder named `react-items-with-hooks` and it will be initialized with a basic React application. Now, you will be able to open the project in your favourite IDE. You can see an `src` folder inside the project along with the main application component `App.js`. This file is having a single function `App()` which will return an element and it will make use of an extended JavaScript syntax(JSX) for defining the component.